

Finding the smallest cyclic shift

The algorithm above sorts all cyclic shifts (without appending a character to the string), and therefore $p[0]$ gives the position of the smallest cyclic shift.

Finding a substring in a string

The task is to find a string s inside some text t online - we know the text t beforehand, but not the string s . We can create the suffix array for the text t in $O(|t| \log |t|)$ time. Now we can look for the substring s in the following way. The occurrence of s must be a prefix of some suffix from t . Since we sorted all the suffixes we can perform a binary search for s in p . Comparing the current suffix and the substring s within the binary search can be done in $O(|s|)$ time, therefore the complexity for finding the substring is $O(|s| \log |t|)$. Also notice that if the substring occurs multiple times in t , then all occurrences will be next to each other in p . Therefore the number of occurrences can be found with a second binary search, and all occurrences can be printed easily.

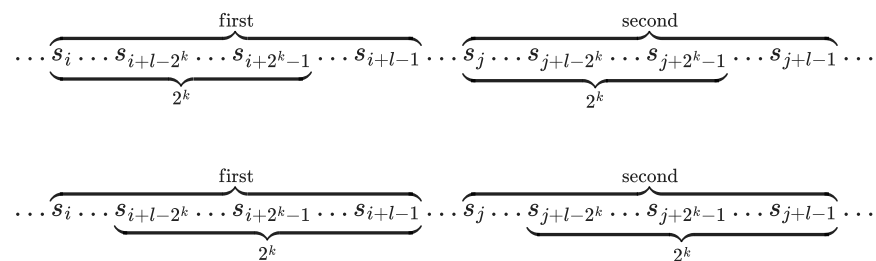
Comparing two substrings of a string

We want to be able to compare two substrings of the same length of a given string s in $O(1)$ time, i.e. checking if the first substring is smaller than the second one.

For this we construct the suffix array in $O(|s| \log |s|)$ time and store all the intermediate results of the equivalence classes $c[]$.

Using this information we can compare any two substring whose length is equal to a power of two in $O(1)$: for this it is sufficient to compare the equivalence classes of both substrings. Now we want to generalize this method to substrings of arbitrary length.

Let's compare two substrings of length l with the starting indices i and j . We find the largest length of a block that is placed inside a substring of this length: the greatest k such that $2^k \leq l$. Then comparing the two substrings can be replaced by comparing two overlapping blocks of length 2^k : first you need to compare the two blocks starting at i and j , and if these are equal then compare the two blocks ending in positions $i + l - 1$ and $j + l - 1$:



Here is the implementation of the comparison. Note that it is assumed that the function gets called with the already calculated k . k can be computed with $\lfloor \log l \rfloor$, but it is more efficient to precompute all k values for every l . See for instance the article about the [Sparse Table](#), which uses a similar idea and computes all log values.

```
int compare(int i, int j, int l, int k) {
    pair<int, int> a = {c[k][i], c[k][(i+l-(1 << k))%n]};
    pair<int, int> b = {c[k][j], c[k][(j+l-(1 << k))%n]};
```

```

    return a == b ? 0 : a < b ? -1 : 1;
}

```

Longest common prefix of two substrings with additional memory

For a given string s we want to compute the longest common prefix (**LCP**) of two arbitrary suffixes with position i and j .

The method described here uses $O(|s| \log |s|)$ additional memory. A completely different approach that will only use a linear amount of memory is described in the next section.

We construct the suffix array in $O(|s| \log |s|)$ time, and remember the intermediate results of the arrays $c[]$ from each iteration.

Let's compute the LCP for two suffixes starting at i and j . We can compare any two substrings with a length equal to a power of two in $O(1)$. To do this, we compare the strings by power of twos (from highest to lowest power) and if the substrings of this length are the same, then we add the equal length to the answer and continue checking for the LCP to the right of the equal part, i.e. i and j get added by the current power of two.

```

int lcp(int i, int j) {
    int ans = 0;
    for (int k = log_n; k >= 0; k--) {
        if (c[k][i % n] == c[k][j % n]) {
            ans += 1 << k;
            i += 1 << k;
            j += 1 << k;
        }
    }
    return ans;
}

```

Here $\log_n$ denotes a constant that is equal to the logarithm of n in base 2 rounded down.

Longest common prefix of two substrings without additional memory

We have the same task as in the previous section. We have compute the longest common prefix (**LCP**) for two suffixes of a string s .

Unlike the previous method this one will only use $O(|s|)$ memory. The result of the preprocessing will be an array (which itself is an important source of information about the string, and therefore also used to solve other tasks). LCP queries can be answered by performing RMQ queries (range minimum queries) in this array, so for different implementations it is possible to achieve logarithmic and even constant query time.

The basis for this algorithm is the following idea: we will compute the longest common prefix for each **pair of adjacent suffixes in the sorted order**. In other words we construct an array $\text{lcp}[0 \dots n - 2]$, where $\text{lcp}[i]$ is equal to the length of the longest common prefix of the suffixes starting at $p[i]$ and $p[i + 1]$. This array will give us an answer for any two adjacent suffixes of the string. Then the answer for arbitrary two suffixes, not necessarily neighboring ones, can be obtained from this array. In fact, let the request be to compute the LCP of the suffixes $p[i]$ and $p[j]$. Then the answer to this query will be $\min(\text{lcp}[i], \text{lcp}[i + 1], \dots, \text{lcp}[j - 1])$.

Thus if we have such an array lcp , then the problem is reduced to the **RMQ**, which has many wide number of different solutions with different complexities.

So the main task is to **build** this array lcp . We will use **Kasai's algorithm**, which can compute this array in $O(n)$ time.

Let's look at two adjacent suffixes in the sorted order (order of the suffix array). Let their starting positions be i and j and their lcp equal to $k > 0$. If we remove the first letter of both suffixes - i.e. we take the suffixes $i + 1$ and $j + 1$ - then it should be obvious that the lcp of these two is $k - 1$. However we cannot use this value and write it in the lcp array, because these two suffixes might not be next to each other in the sorted order. The suffix $i + 1$ will of course be smaller than the suffix $j + 1$, but there might be some suffixes between them. However, since we know that the LCP between two suffixes is the minimum value of all transitions, we also know that the LCP between any two pairs in that interval has to be at least $k - 1$, especially also between $i + 1$ and the next suffix. And possibly it can be bigger.

Now we already can implement the algorithm. We will iterate over the suffixes in order of their length. This way we can reuse the last value k , since going from suffix i to the suffix $i + 1$ is exactly the same as removing the first letter. We will need an additional array rank , which will give us the position of a suffix in the sorted list of suffixes.

```
vector<int> lcp_construction(string const& s, vector<int> const& p) {
    int n = s.size();
    vector<int> rank(n, 0);
    for (int i = 0; i < n; i++)
        rank[p[i]] = i;

    int k = 0;
    vector<int> lcp(n-1, 0);
    for (int i = 0; i < n; i++) {
        if (rank[i] == n - 1) {
            k = 0;
            continue;
        }
        int j = p[rank[i] + 1];
        while (i + k < n && j + k < n && s[i+k] == s[j+k])
            k++;
        lcp[rank[i]] = k;
        if (k)
            k--;
    }
    return lcp;
}
```

It is easy to see, that we decrease k at most $O(n)$ times (each iteration at most once, except for $\text{rank}[i] == n - 1$, where we directly reset it to 0), and the LCP between two strings is at most $n - 1$, we will also increase k only $O(n)$ times. Therefore the algorithm runs in $O(n)$ time.

Number of different substrings

We preprocess the string s by computing the suffix array and the LCP array. Using this information we can compute the number of different substrings in the string.

To do this, we will think about which **new** substrings begin at position $p[0]$, then at $p[1]$, etc. In fact we take the suffixes in sorted order and see what prefixes give new substrings. Thus we will not overlook any by accident.

Because the suffixes are sorted, it is clear that the current suffix $p[i]$ will give new substrings for all its prefixes, except for the prefixes that coincide with the suffix $p[i - 1]$. Thus, all its prefixes except the first $\text{lcp}[i - 1]$ one. Since the length of the current suffix is $n - p[i]$, $n - p[i] - \text{lcp}[i - 1]$ new prefixes start at $p[i]$. Summing over all the suffixes, we get the final answer:

$$\sum_{i=0}^{n-1} (n - p[i]) - \sum_{i=0}^{n-2} \text{lcp}[i] = \frac{n^2 + n}{2} - \sum_{i=0}^{n-2} \text{lcp}[i]$$

Practice Problems

- [Uva 760 - DNA Sequencing](#)
- [Uva 1223 - Editor](#)
- [Codechef - Tandem](#)
- [Codechef - Substrings and Repetitions](#)
- [Codechef - Entangled Strings](#)
- [Codeforces - Martian Strings](#)
- [Codeforces - Little Elephant and Strings](#)
- [SPOJ - Ada and Terramorphing](#)
- [SPOJ - Ada and Substring](#)
- [UVA - 1227 - The longest constant gene](#)
- [SPOJ - Longest Common Substring](#)
- [UVA 11512 - GATTACA](#)
- [LA 7502 - Suffixes and Palindromes](#)
- [GYM - Por Costel and the Censorship Committee](#)
- [UVA 1254 - Top 10](#)
- [UVA 12191 - File Recover](#)
- [UVA 12206 - Stammering Aliens](#)
- [Codechef - Jarvis and LCP](#)
- [LA 3943 - Liking's Letter](#)
- [UVA 11107 - Life Forms](#)
- [UVA 12974 - Exquisite Strings](#)
- [UVA 10526 - Intellectual Property](#)
- [UVA 12338 - Anti-Rhyme Pairs](#)
- [UVA 12191 - File Recover](#)
- [SPOJ - Suffix Array](#)
- [LA 4513 - Stammering Aliens](#)
- [SPOJ - LCS2](#)
- [Codeforces - Fake News \(hard\)](#)
- [SPOJ - Longest Common Substring](#)
- [SPOJ - Lexicographical Substring Search](#)